

RETAILMART V3 • SQL

Database Concepts for Analysts

WhatsApp Announcement

Database Concepts for Analysts

Today is different -- no new query syntax. Instead, we learn the THEORY behind why databases are designed the way they are. These concepts show up in every data analyst interview and explain why RetailMart's tables look the way they do.

Part 1: Normalization (1NF, 2NF, 3NF) -- why tables are split the way they are

Part 2: ACID Properties -- the 4 guarantees every database makes

Part 3: Transactions -- BEGIN, COMMIT, ROLLBACK

When an interviewer asks 'Why is this data spread across 5 tables instead of one big table?' -- today gives you the answer. Every data analyst must understand normalization and ACID, even though you will never design schemas yourself.

Prerequisites: All SQL from earlier sessions. Today we step back and understand the WHY behind database design.

Recap

Last time we completed our subquery and CTE toolkit:

- Correlated Subqueries -- inner query references the outer row
- EXISTS / NOT EXISTS -- NULL-safe existence checks
- CTEs (WITH clause) -- named pipeline steps replacing nested subqueries
- Recursive CTEs -- traverse hierarchies and generate series

Today we pause new syntax and learn WHY databases are designed the way they are. Understanding normalization, ACID, and transactions makes you a stronger analyst and better interview candidate.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: Normalization	50 min	1NF, 2NF, 3NF with examples, Functional dependencies, Denormalization tradeoffs in RetailMart
Part 2: ACID Properties	30 min	Atomicity, Consistency, Isolation, Durability, Plain-English meaning with one example each, Which matters most for analysts?
Part 3: Transactions	40 min	BEGIN, COMMIT, ROLLBACK syntax, Why your long SELECT is inside a transaction, READ COMMITTED isolation level

YOUR SQL JOURNEY

- ✓ Conditional Logic
- ✓ Aggregates
- ✓ Joins Part 1
- ✓ Joins Part 2
- ✓ Self Join & UNION
- ✓ Subqueries Part 1
- ✓ Subqueries Part 2 + CTEs

The Intern's First Database Design Review (1/2)

A new data engineering intern at RetailMart proposes a brilliant idea: 'Why do we have 55 tables? Let us put EVERYTHING in one giant table -- customer name, order details, product info, store address, all in one row. No JOINS needed. Queries will be so much faster.'

The senior engineer smiles. She has seen this proposal before. She opens a spreadsheet with 150,000 order rows and asks three questions:

Question 1: 'Customer Priya Sharma got married and changed her last name. How many rows do you need to update?' In one giant table: every single order she ever placed. Miss one row and now Priya has two different names in the database.

Question 2: 'We want to add a new product that nobody has ordered yet. Can you add it?' In one giant table: no. Every row needs an order. A product with zero orders cannot exist.

Question 3: 'A cashier accidentally deletes the last order for a product. What happens to the product information?' In one giant table: the product vanishes completely. Its name, price, brand -- all gone.

The Intern's First Database Design Review (2/2)

These are the three anomalies -- Update, Insert, Delete -- that normalization prevents. By the end of today, you will understand exactly WHY RetailMart uses 55 tables instead of one.

From One Messy Spreadsheet to Clean Tables

Imagine an Excel spreadsheet tracking orders at a small shop. Each row has: OrderID, Date, CustomerName, CustomerPhone, ProductName, ProductPrice, Quantity, StoreName, StoreCity.

Customer 'Rahul Verma' places 50 orders. His phone number appears 50 times. One day he changes his phone. You update 49 of the 50 rows. Now your database says Rahul has TWO different phone numbers. Which one is correct? Nobody knows.

Normalization is the process of splitting this one messy table into multiple clean tables -- each fact stored exactly once. Rahul's phone number lives in ONE row in the customers table. Orders reference him by customer_id. Change his phone once, it is correct everywhere.

First Normal Form (1NF) (1/2)

Rule:

Every cell contains a single atomic value. No repeating groups. No arrays, no comma-separated lists, no nested tables inside cells.

Violation example:

A products column that stores 'Laptop, Mouse, Keyboard' as a single comma-separated string. Or a phone column with '9876543210, 9123456789' for two numbers.

Fix:

One row per product per order (order_items table). One row per phone number if a customer has multiple phones. Each cell = one value.

First Normal Form (1NF) (2/2)

RetailMart example:

`sales.order_items` stores one row per product per order. If an order has 3 products, that is 3 rows in `order_items`, not one row with a comma-separated product list.

1NF Violation vs 1NF Compliant

VIOLATES 1NF (comma-separated products):

order_id	products	quantities
1001	Laptop, Mouse, Cable	1, 2, 1

1NF COMPLIANT (one value per cell):

order_id	product	quantity
1001	Laptop	1
1001	Mouse	2
1001	Cable	1

```
-- RetailMart: sales.order_items is exactly this
SELECT order_id, product_id, quantity, unit_price
FROM sales.order_items
WHERE order_id = 1001;
```

Second Normal Form (2NF) (1/2)

Prerequisite:

Table must already be in 1NF.

Rule:

Every non-key column must depend on the ENTIRE primary key, not just part of it. This only matters for tables with composite primary keys (two or more columns forming the PK).

Violation example:

A table with PK = (order_id, product_id) that also stores customer_name. Customer name depends only on order_id (which links to the customer), not on product_id. It is a partial dependency.

Second Normal Form (2NF) (2/2)

Fix:

Move customer_name to the orders table (where it depends on the full PK). The order_items table keeps only columns that depend on BOTH order_id AND product_id (like quantity and unit_price).

RetailMart example:

sales.order_items has (order_item_id) as PK but logically depends on (order_id, product_id). It stores quantity and unit_price -- both depend on which product in which order. Customer info is NOT here -- it lives in sales.orders.

2NF Violation vs 2NF Compliant

VIOLATES 2NF (customer_name depends only on order_id):

order_id	product_id	qty	customer_name	
1001	42	2	Priya Sharma	
1001	87	1	Priya Sharma	<- repeated!
1002	42	3	Rahul Verma	

2NF COMPLIANT (split into two tables):

orders:	order_id	cust_id	
	1001	501	
order_items:	order_id	product_id	qty
	1001	42	2
	1001	87	1
customers:	cust_id	name	
	501	Priya Sharma	

Third Normal Form (3NF) (1/2)

Prerequisite:

Table must already be in 2NF.

Rule:

No transitive dependencies. Every non-key column must depend DIRECTLY on the primary key, not on another non-key column.

Violation example:

An employees table with PK = employee_id, columns: name, store_id, store_city. Store_city depends on store_id, not on employee_id. If Store 5 moves from Mumbai to Pune, you must update every employee at Store 5.

Third Normal Form (3NF) (2/2)

Fix:

Move store_city to a stores table. The employees table stores store_id (FK). Store_city is looked up via JOIN when needed.

RetailMart example:

stores.employees has store_id but NOT store_city. The city lives in stores.stores. JOIN when needed. Each fact stored exactly once.

3NF Violation vs 3NF Compliant

VIOLATES 3NF (store_city depends on store_id, not employee_id):

emp_id	name	store_id	store_city	
1	Alice	5	Mumbai	
2	Bob	5	Mumbai	<- repeated!
3	Carol	8	Delhi	

3NF COMPLIANT (city lives with store):

employees:	emp_id	name	store_id
	1	Alice	5

stores:	store_id	city
	5	Mumbai

-- RetailMart verifies this:

```
SELECT e.first_name, s.city
FROM stores.employees e
JOIN stores.stores s ON e.store_id = s.store_id;
```

Functional Dependencies -- The Formal Foundation (1/2)

A functional dependency means: if you know column A, you can determine column B. Written as $A \rightarrow B$ ('A determines B').

Examples from RetailMart:

- $\text{customer_id} \rightarrow \text{first_name}, \text{email}, \text{phone}$ (knowing the customer ID determines their details)
- $\text{store_id} \rightarrow \text{store_name}, \text{city}, \text{state}$ (knowing the store ID determines its location)
- $\text{order_id} \rightarrow \text{cust_id}, \text{order_date}, \text{net_total}$ (knowing the order ID determines order details)

Normalization rules are really about functional dependencies: 2NF says 'non-key columns must depend on the FULL key.' 3NF says 'non-key columns must depend ONLY on the key, not on other non-key columns.'

REVEAL

Functional Dependencies -- The Formal Foundation (2/2)

The memory trick: 'Every non-key column depends on the key, the whole key, and nothing but the key -- so help me Codd.' (Edgar F. Codd invented normalization in 1970.)

When Breaking the Rules Is the Right Call

RetailMart's sales.orders table has a total_amount column. But this value can be COMPUTED from order_items: $\text{SUM}(\text{unit_price} * \text{quantity})$. So total_amount is redundant -- it violates 3NF (it depends on order_items, not just on order_id).

So why does RetailMart keep it? Because computing the total from 350,000 order_items rows every time an analyst runs a revenue query is SLOW. Storing the pre-computed total is a deliberate tradeoff: trade some redundancy for massive read performance.

This is denormalization -- intentionally adding redundancy to speed up reads. Warehouses and analytics databases denormalize heavily. Transaction databases (banks, inventory) normalize strictly.

The rule: Normalize first. Denormalize with intent, never by accident. Always document what is denormalized and why.

Lab: Audit RetailMart for Normalization

HARD

SCENARIO: Your manager asks you to audit RetailMart's schema. For each table listed below, determine: is it in 3NF, or does it contain denormalized (redundant/derived) fields? If denormalized, explain WHY the design team chose to keep those fields.

TASK: Examine these 5 tables. For each, state whether it is 3NF or denormalized, identify any redundant columns, and explain the tradeoff.

Tables to audit:

- sales.orders (has net_total, discount -- can these be derived?)
- sales.order_items (has unit_price -- does product price change over time?)
- stores.employees (has store_id -- is any store info duplicated here?)
- products.products (has supplier_id + supplier info or just the FK?)
- finance.revenue_summary (is this entire table derived from orders?)

HINT: For each column ask two questions in sequence — 'Can I COMPUTE this from another table?' If yes, it's denormalized. 'Is the redundancy justified for read speed?' If yes, it's a deliberate trade-off. Trap: a column that LOOKS redundant might actually be a HISTORICAL SNAPSHOT (like order_items.unit_price capturing the price at purchase time, not the current price) — don't flag those as denormalization.

Answer

✓ ANSWER

```
-- 1. sales.orders: DENORMALIZED
-- net_total / gross_total / discount_amount
-- could be derived from order_items aggregation
-- Justified: avoids joining 350K items for every
-- revenue dashboard query

-- 2. sales.order_items: 3NF (unit_price NOT redundant!)
-- unit_price captures the price AT TIME OF PURCHASE
-- Product price may change later; this is a snapshot

-- 3. stores.employees: 3NF
-- Only store_id (FK), no store_name/city duplicated
-- JOIN to stores.stores for location details

-- 4. products.products: 3NF
-- Only supplier_id (FK), no supplier name/contact
-- JOIN to products.suppliers for supplier details

-- 5. finance.revenue_summary: FULLY DENORMALIZED
-- Entire table derived from sales.orders aggregation
-- Justified: pre-computed monthly summaries
-- Tradeoff: must be refreshed when orders change
```

KEY TAKEAWAYS

1NF: Every cell has one atomic value (no arrays/lists). 2NF: Non-key columns depend on the WHOLE key (no partial dependencies). 3NF: Non-key columns depend ONLY on the key (no transitive dependencies). Memory trick: 'The key, the whole key, and nothing but the key.' Denormalization is intentionally breaking these rules for read performance -- common in analytics databases.

PRO TIP

PRO TIP

As an analyst, you will never design schemas. But you will QUERY them. Understanding normalization tells you WHERE data lives. If you need a customer's city, do not look in the orders table -- it is in the customers or stores table. Normalization is your mental map for navigating any database.

Common Misconceptions about Normalization

Misconception 1: 'More tables = slower queries'

JOINS are optimized by the database engine. 5 normalized tables with proper indexes often outperform 1 giant denormalized table because the engine reads less data.

Reality:

Normalized schemas with indexes are fast for most queries. Denormalize only when you have measured a real performance problem.

Misconception 2: 'Analysts should always normalize'

Analytics warehouses are often denormalized (star schemas, wide fact tables). The write penalty of denormalization does not matter when data is loaded in batch, not written row-by-row.

Reality:

Normalize for OLTP (transaction processing). Denormalize for OLAP (analytics). Know both, apply the right one.

What Happens When a Payment Crashes Midway?

A customer buys a laptop for 75,000. The payment system must do two things: (1) deduct 75,000 from the customer's wallet, (2) add 75,000 to RetailMart's revenue account.

The server crashes AFTER step 1 but BEFORE step 2. The customer lost 75,000. RetailMart never received it. The money vanished into thin air.

ACID properties prevent this. Atomicity says: either BOTH steps happen, or NEITHER happens. If the server crashes after step 1, the database rolls back step 1 automatically. The customer's 75,000 is restored.

Every database you will ever use as an analyst provides ACID guarantees. Understanding them explains why your data is trustworthy.

A -- Atomicity (All or Nothing)

Meaning:

A transaction is an indivisible unit of work. Either ALL operations in the transaction complete successfully, or NONE of them do. There is no in-between state.

Example:

Transfer 75,000 from customer wallet to store revenue: (1) Deduct from wallet. (2) Add to revenue. If step 2 fails, step 1 is automatically undone (rolled back). The money never disappears.

Analyst impact:

When you see an order in sales.orders, you know the complete order was recorded -- not a partial order with missing items. The payment, shipment, and inventory updates either all happened or none did.

C -- Consistency (Rules Are Never Broken)

Meaning:

A transaction moves the database from one valid state to another valid state. All constraints (CHECK, FK, UNIQUE, NOT NULL) must be satisfied before and after every transaction.

Example:

A CHECK constraint says $\text{price} > 0$. A transaction that tries to set $\text{price} = -500$ will be rejected entirely. The database stays in a valid state.

Analyst impact:

When you query `products.products`, you know every price is positive. You do not need to add `WHERE price > 0` as a safety net -- the database GUARANTEES it. This is why constraints from Constraints & DML matter.

I -- Isolation (Transactions Do Not Interfere)

Meaning:

Concurrent transactions execute as if they were running one at a time. One transaction's intermediate state is invisible to others.

Example:

Two cashiers process orders at the same time. Cashier A's half-completed order is invisible to Cashier B's transaction. Each sees a consistent view of the database.

Analyst impact:

When you run a long analytics query (say, 30 seconds), the data does not change underneath you mid-query. Your SELECT sees a consistent snapshot. This is why your results are reliable even on a busy production database.

D -- Durability (Committed = Permanent)

Meaning:

Once a transaction is committed (COMMIT succeeds), the data is permanently saved, even if the server crashes, power goes out, or the disk fails (with proper backups).

Example:

A customer places an order at 3:47 PM. The transaction commits. At 3:48 PM the server crashes and reboots. The order is still there when the server comes back up. Committed data is written to disk (WAL -- Write-Ahead Log).

Analyst impact:

When you query yesterday's data, you can trust that no committed transactions were lost overnight. Your historical analyses are accurate.

ACID Summary -- Quick Reference

A - Atomicity: All or nothing. No partial transactions.
C - Consistency: Rules (constraints) always hold.
I - Isolation: Concurrent users see consistent snapshots.
D - Durability: Committed data survives crashes.

-- Which matters most for an analyst?
-- ISOLATION: guarantees your long SELECT sees
-- consistent data even while others INSERT/UPDATE
-- CONSISTENCY: guarantees constraint rules hold
-- (no negative prices, no orphan FKs)

Medium: Rank ACID Properties for Analysts

MEDIUM

SCENARIO: Your manager asks: 'As a data analyst, which ACID property affects your daily work the most?' She wants you to rank all four from most to least impactful and explain each ranking. This is a common interview question for analytics roles.

TASK: Rank the 4 ACID properties by how much they affect an analyst's daily work. For each, give one concrete example from your work querying RetailMart.

HINT: Think about what you do every day: run long SELECT queries, trust historical data, assume constraints hold, never worry about partial records.

Answer



```
-- Suggested ranking (analyst perspective):

-- 1. ISOLATION (most impactful)
-- Your 30-second revenue query sees a consistent
-- snapshot even while 100 new orders are being inserted.
-- Without isolation, your SUM(net_total) could include
-- half-inserted orders = wrong numbers for the CFO.

-- 2. CONSISTENCY
-- Every FK is valid, every CHECK holds. You trust that
-- orders.store_id always points to a real store.
-- Without it, JOINS would silently drop rows.

-- 3. DURABILITY
-- Yesterday's data is still there today. Your weekly
-- trend report does not lose a random Wednesday.

-- 4. ATOMICITY (least daily impact for analysts)
-- Prevents partial records. Important but invisible --
-- you benefit from it without ever thinking about it.
```

KEY TAKEAWAYS

ACID = Atomicity (all or nothing), Consistency (rules hold), Isolation (no interference), Durability (committed = permanent). For analysts, Isolation matters most: your long queries see consistent snapshots. Consistency matters next: constraints guarantee data quality. These are guarantees, not settings -- every PostgreSQL transaction gets all four automatically.

PRO TIP

PRO TIP

In interviews, do not just recite the definitions. Give examples. 'Atomicity means a bank transfer either fully completes or fully rolls back -- no money disappears.' Show you UNDERSTAND the concept, not just memorized it.

The Accidental DELETE

A junior DBA at RetailMart runs: `DELETE FROM customers.customers WHERE city = 'Mumbai'`. He meant to delete from a test table. 12,000 Mumbai customers vanish instantly.

If he had wrapped the command in a transaction (`BEGIN ... ROLLBACK`), he could have undone it. Without a transaction wrapper, PostgreSQL auto-commits every statement. The DELETE was permanent the moment it executed.

As analysts, you will rarely write DELETE. But you WILL write complex UPDATE and INSERT statements during data cleaning. Wrapping risky operations in transactions is your safety net.

Transactions -- BEGIN, COMMIT, ROLLBACK (1/2)

BEGIN:

Starts a transaction block. Nothing is finalized until COMMIT.

COMMIT:

Finalizes all changes made since BEGIN. Once committed, changes are permanent (Durability).

ROLLBACK:

Undoes all changes made since BEGIN. The database returns to its state before BEGIN. Use this when something goes wrong mid-transaction.

Transactions -- BEGIN, COMMIT, ROLLBACK (2/2)

Auto-commit:

Without an explicit BEGIN, PostgreSQL wraps every single statement in its own tiny transaction and auto-commits it. This is why a bare DELETE is immediately permanent.

Transaction Syntax

```
-- Safe pattern: BEGIN, check, then COMMIT or ROLLBACK  
BEGIN;
```

```
-- Make changes  
UPDATE products.products  
SET price = price * 1.10  
WHERE category = 'Electronics';
```

```
-- Check before committing:  
SELECT product_name, price  
FROM products.products  
WHERE category = 'Electronics'  
LIMIT 5;
```

```
-- If results look correct:  
COMMIT;    -- changes are now permanent
```

```
-- If something looks wrong:  
ROLLBACK;  -- undo everything since BEGIN
```

Your SELECT Is Already Inside a Transaction

When you run a long analytics query, PostgreSQL automatically wraps it in an implicit transaction. This is how Isolation works in practice.

Example: You run `SELECT SUM(net_total) FROM sales.orders`. This query takes 15 seconds. During those 15 seconds, a cashier inserts 20 new orders. Does your SUM include those 20 new orders?

Answer: NO. Your query sees a snapshot from the moment it started. The 20 new orders are invisible to your transaction. This is PostgreSQL's default isolation level: `READ COMMITTED`.

`READ COMMITTED` means: each statement in your transaction sees only data that was committed BEFORE the statement started. Uncommitted changes from other transactions are invisible.

This is why your analytics results are trustworthy on a busy production database. You are always reading a consistent snapshot.

Implicit Transaction -- What You Do Not See

```
-- What you type:
SELECT SUM(net_total)
FROM sales.orders
WHERE order_date >= '2025-01-01';

-- What PostgreSQL actually does:
-- BEGIN; (implicit)
-- Take snapshot of committed data
-- Execute your SELECT against that snapshot
-- COMMIT; (implicit, read-only = no changes to commit)

-- Even if 100 new orders are inserted while your
-- query runs, your result is based on the snapshot.
-- PostgreSQL default: READ COMMITTED isolation level
```

Easy: Spot the Normalization Violations

EASY

SCENARIO: A new intern designed three tables for a side-project (NOT RetailMart). Each table has a normalization violation. Identify the normal form being violated and explain the fix. The patterns map directly to why RetailMart has separate tables for products vs brands, customers vs addresses, etc.

TASK: For each table below, name the violated normal form (1NF, 2NF, or 3NF) and describe how to fix it.

Table 1: campaigns_bad -- columns: campaign_id, campaign_name, platforms ('Google, Meta, YouTube' in one column)

Table 2: ticket_assignments -- PK: (ticket_id, agent_id), columns: priority, agent_email (email depends only on agent_id)

Table 3: orders_flat -- PK: order_id, columns: product_id, product_name, product_brand (brand depends on product_id, not order_id)

HINT: Three rules to remember by the mnemonic 'key / whole key / nothing but the key' — atomic cells, full dependency on composite key, no transitive dependencies. Trap: 2NF only matters when there's a COMPOSITE primary key; if your PK is a single column, every functional dependency on it is automatically 'on the whole key' — 2NF violations are impossible there.

Answer (1/2)



```
-- Table 1: VIOLATES 1NF
-- 'Google, Meta, YouTube' = multi-valued cell
-- Fix: Create campaign_platforms junction table
-- | campaign_id | platform |
-- | 1           | Google  |
-- | 1           | Meta    |
-- RetailMart parallel: marketing.ads_spend
-- has ONE platform per row, not a comma list

-- Table 2: VIOLATES 2NF
-- agent_email depends only on agent_id
--   (partial dependency on composite PK)
-- Fix: Move email to the agents (employees) table
-- RetailMart parallel: support.tickets has
-- agent_id (FK) – email lives in stores.employees

-- Table 3: VIOLATES 3NF
-- product_brand depends on product_id,
--   not on order_id (transitive dependency)
-- Fix: Move brand to products; orders.order_items
--   references product_id via FK
-- RetailMart parallel: sales.order_items stores
```

Answer (2/2)

✓ ANSWER

```
-- prod_id ONLY – brand info is in products + dim_brand
```

Medium: Design a Many-to-Many Junction Table

MEDIUM

SCENARIO: RetailMart wants to track which promotions apply to which products. One promotion can apply to many products. One product can be part of many promotions. This is a many-to-many (M:N) relationship.

TASK: Design a junction table called `promotion_products`. Specify its columns, primary key, and foreign keys. Explain why this table is necessary and what normalization problem it solves.

HINT: A many-to-many relationship can't be expressed with a single FK — neither side has 'one parent'. The bridge pattern uses a third table holding both FKs with a composite PK. Trap: forgetting the composite primary key on (`promo_id`, `product_id`) lets the same `promo`×`product` pair be inserted multiple times, silently duplicating discounts on the same product.

Answer



```
-- Junction table design:
CREATE TABLE promotion_products (
    promotion_id INT REFERENCES products.promotions(promo_id),
    product_id   INT REFERENCES products.products(product_id),
    discount_pct NUMERIC(5,2), -- per-product override
    PRIMARY KEY (promotion_id, product_id)
);
```

```
-- Without this table, you would need:
-- Option A: comma-separated product_ids in promotions
--   -> VIOLATES 1NF
-- Option B: duplicate promotion rows per product
--   -> VIOLATES 2NF (promo name depends only on promo_id)

-- The junction table is the only 3NF solution for M:N
```

Hard: Argue For/Against Denormalizing a RetailMart Table

HARD

SCENARIO: The analytics team proposes adding a `customer_name` column directly to `sales.orders` (currently you JOIN to `customers.customers` to get it). They argue it will speed up their most common query by eliminating a JOIN on 150,000 rows.

TASK: Write a 2-paragraph argument: one paragraph FOR adding `customer_name` to orders (the benefits), one paragraph AGAINST (the risks). Then make your recommendation with justification.

HINT: Denormalization is a TRADE — read speed in exchange for write complexity. Trap: the 'two sources of truth' problem isn't theoretical — it bites when a customer changes their name and one place gets updated but the other doesn't, leaving the same person spelled two ways in the same database.

Answer (1/2)

✓ ANSWER

```
-- FOR denormalization:
-- 1. The most-run analyst query is 'orders with customer name'
--    Eliminating the JOIN saves ~200ms per query
-- 2. 80% of ad-hoc queries need customer_name
--    Adding it to orders simplifies every query
-- 3. Customer names rarely change (maybe 0.1% per year)

-- AGAINST denormalization:
-- 1. Update anomaly: if Priya changes her name,
--    you must update 150,000 order rows (her orders)
-- 2. Storage: 50 bytes x 150K rows = 7.5 MB wasted
-- 3. Two sources of truth: which is correct --
--    orders.customer_name or customers.first_name?
-- 4. Every INSERT must now populate customer_name too

-- RECOMMENDATION: Do NOT denormalize.
-- Instead, create a VIEW that pre-joins:
CREATE VIEW sales.v_orders_with_customer AS
SELECT o.*, c.first_name, c.last_name
FROM sales.orders o
JOIN customers c
  ON o.cust_id = c.customer_id;
```

Answer (2/2)

 ANSWER

-- Views give JOIN-free syntax WITHOUT redundancy

Crazy: Write a Safe Wallet-to-Wallet Transfer

CRAZY

SCENARIO: This is THE canonical transaction use-case. A customer transfers points from one wallet to another. Two operations must succeed together: DEBIT from sender, CREDIT to receiver. If only one runs, money vanishes or appears from nowhere. Use `customers.wallets` and `finance.transfer_log` in your PRACTICE DB (RetailMart V3 is read-only).

TASK: Write a transaction block that: (1) BEGIN the transaction, (2) verify both wallet balances, (3) DEBIT sender, (4) CREDIT receiver, (5) INSERT a `transfer_log` entry, (6) verify final balances, (7) show COMMIT and ROLLBACK options with comments. This is atomic — all five steps succeed or none do.

HINT: The textbook transaction pattern — debit + credit + log entry must all succeed or all roll back. Trap: forgetting the WHERE clause on the UPDATE drops every wallet to the same balance — always test transactions in your practice DB before approving similar changes in production; ROLLBACK is your safety net.

Answer (1/2)

✓ ANSWER

```
-- Run this in your accio_<batch> practice DB
-- NOT on RetailMart (which is read-only)
```

```
-- STEP 1: Start transaction
BEGIN;
```

```
-- STEP 2: Verify current balances
SELECT cust_id, balance
FROM customers.wallets
WHERE cust_id IN (101, 202);
```

```
-- STEP 3: Debit sender (cust 101)
UPDATE customers.wallets
SET balance = balance - 5000,
    last_updated = NOW()
WHERE cust_id = 101;
```

```
-- STEP 4: Credit receiver (cust 202)
UPDATE customers.wallets
SET balance = balance + 5000,
    last_updated = NOW()
WHERE cust_id = 202;
```

Answer (2/2)

✓ ANSWER

```
-- STEP 5: Log the transfer (audit trail)
INSERT INTO finance.transfer_log
    (from_account, to_account,
     amount, transferred_at, status)
VALUES
    (101, 202, 5000, NOW(), 'Completed');

-- STEP 6: Verify final balances
SELECT cust_id, balance
FROM customers.wallets
WHERE cust_id IN (101, 202);

-- STEP 7: Finalize
-- All four ops look correct?
COMMIT;
-- Balance went negative or wrong account?
-- ROLLBACK; -- all 4 ops undone atomically
```

What is Normalization?

Q: Explain normalization and its normal forms.

A: Normalization organizes a database to reduce redundancy and prevent anomalies. 1NF: every cell is atomic (no arrays). 2NF: non-key columns depend on the entire composite key (no partial dependencies). 3NF: non-key columns depend only on the primary key (no transitive dependencies). The goal is: each fact stored exactly once. The memory trick: 'The key, the whole key, and nothing but the key.'

Normalization vs Denormalization

Q: When would you denormalize a database?

A: Denormalize when read performance matters more than write correctness -- typically in analytics warehouses, reporting databases, and data marts. OLTP systems (banks, inventory) should stay normalized to prevent anomalies. Common denormalization: storing pre-computed aggregates (total_amount on orders), adding redundant columns to avoid frequent JOINS. Always normalize first, then denormalize with intent and documentation.

What is ACID?

Q: Explain the ACID properties of a database.

A: ACID guarantees reliable transactions. Atomicity: all-or-nothing execution. Consistency: constraints always hold. Isolation: concurrent transactions do not interfere (each sees a consistent snapshot). Durability: committed data survives crashes. Example: a bank transfer debits one account and credits another. Atomicity ensures both happen or neither does. Isolation ensures another query does not see the intermediate state where money has left one account but not arrived at the other.

What is a Transaction?

Q: What are BEGIN, COMMIT, and ROLLBACK?

A: BEGIN starts a transaction block -- a group of SQL statements treated as one unit. COMMIT finalizes all changes permanently. ROLLBACK undoes all changes since BEGIN. Without explicit BEGIN, PostgreSQL auto-commits each statement individually. Transactions are essential for multi-step operations where partial completion would leave the database in an inconsistent state.

REAL WORLD (1/2)

What You Achieved Today

1. Normalization (1NF, 2NF, 3NF)

1NF: atomic cells, no multi-valued fields. 2NF: no partial dependency on composite keys. 3NF: no transitive dependencies through non-key columns. Denormalization is the intentional reverse -- adding redundancy for read speed. RetailMart uses both: normalized base tables + denormalized summaries.

2. ACID Properties

Atomicity (all or nothing), Consistency (constraints hold), Isolation (snapshot reads), Durability (committed = permanent). For analysts, Isolation is king -- your long queries always see consistent data.

3. Transactions

REAL WORLD (2/2)

BEGIN/COMMIT/ROLLBACK for safe multi-step changes. Auto-commit wraps every bare statement. Your SELECT queries implicitly run inside transactions with READ COMMITTED isolation.

Why This Matters:

- Interviews: Normalization and ACID are asked in 70%+ of data analyst interviews
- Schema navigation: Understanding 3NF tells you WHERE data lives across tables
- Data trust: ACID guarantees explain why your query results are reliable
- Safe operations: Transactions prevent accidental data destruction

Next up (Review & Practice): Review & Practice -- 8 mixed-difficulty problems spanning everything from earlier sessions. No new concepts. Pure reinforcement and speed practice.

Database Concepts for Analysts

-- 1NF -- Atomic Values

```
-- NO: 'Math, Science' in one cell
-- YES: one row per subject
| student_id | subject |
| 1          | Math    |
| 1          | Science |
```

-- 2NF -- Full Key Dependency

```
-- PK: (order_id, product_id)
-- NO: customer_email here (partial dep)
-- YES: only qty, unit_price
-- Move email to orders/customers table
```

-- 3NF -- No Transitive Deps

```
-- PK: employee_id
-- NO: store_city here (depends on store_id)
-- YES: store_id only (FK)
-- JOIN stores.stores for city
```

Database Concepts for Analysts

-- ACID Quick Reference

A - Atomicity: All or nothing
C - Consistency: Constraints hold
I - Isolation: Snapshot reads
D - Durability: Committed = permanent

-- Transaction Syntax

```
BEGIN;  
-- your changes here  
-- inspect results  
COMMIT;    -- finalize  
-- or --  
ROLLBACK;  -- undo everything
```

-- Denormalization Tradeoffs

```
-- PRO: faster reads (no JOIN)  
-- CON: update anomalies  
-- CON: two sources of truth  
-- Rule: normalize first,  
--       denormalize with intent
```

CHALLENGE (1/3)

8 Practice Problems

Normalization (1-4)

- 1.: Given a table: orders(order_id, customer_name, customer_phone, product_list, total). Identify ALL normal form violations and show the normalized schema (3NF).
- 2.: Examine RetailMart's stores.employees table. Is it in 3NF? List every column and what it depends on. Does any column create a transitive dependency?
- 3.:

CHALLENGE (2/3)

Design a many-to-many relationship: employees can have multiple skills, and each skill can belong to multiple employees. Create the junction table with proper PKs and FKs.

4.: The finance team wants to add a 'store_name' column directly to finance.expenses to avoid JOINing with stores.stores. Write a 3-sentence argument for or against, citing specific normalization rules.

ACID & Transactions (5-8)

5.: A bank transfer moves 10,000 from Account A to Account B. Which ACID property prevents the money from disappearing if the server crashes between debit and credit?

6.:

CHALLENGE (3/3)

You run a 30-second revenue report. During those 30 seconds, a cashier inserts 50 new orders. Does your report include those orders? Name the ACID property and PostgreSQL isolation level that determines the answer.

7.: Write a safe transaction block that deletes all orders with status = 'Cancelled' from before 2024. Include a verification step and both COMMIT/ROLLBACK options.

8.: (Discussion) If PostgreSQL crashed and rebooted, how does it know which transactions were committed and which to roll back? Research 'Write-Ahead Log (WAL)' and explain in 2-3 sentences.

YOUR SQL JOURNEY

- ✓ Conditional Logic
- ✓ Aggregates
- ✓ Joins Part 1
- ✓ Joins Part 2
- ✓ Self Join & UNION
- ✓ Subqueries Part 1
- ✓ Subqueries Part 2 + CTEs